



AirScout: Tactical Cavalry Scout Autonomous UAV System

Platform macOS ARM64 ROS 2 Jazzy Jalisco Simulator Gazebo Harmonic Autopilot PX4 SITL
Vision Ultralytics YOLOv10

AirScout is an advanced, fully integrated autonomous Unmanned Aerial Vehicle (UAV) software stack synthesized for tactical reconnaissance and "Cavalry Scout" missions. Deploying a custom quadcopter drone (`x500_depth_0`) equipped with an RGB-D camera sensor (OakD-Lite simulation) in a complex, forested hill terrain environment (`baylands_custom`), the system autonomously navigates, avoids obstacles, detects target clusters, and projects visual findings onto geographical ground coordinates.

The system is managed through a built-in real-time HTTP Web Cockpit Dashboard, allowing operators to execute mission doctrines, monitor flights, and view consolidated contact reports.



Repository

The source code for this project is hosted on GitHub:

<https://github.com/felix93ie/Robotics>

[Context] **Squad-Level Abstraction:** AirScout eliminates the need for highly trained drone pilots. By abstracting away flight mechanics and active collision avoidance behind simple military grid coordinate inputs (MGRS) and click-to-execute tactics, the system can be deployed directly by any infantry squad or platoon element with near-zero training.

[Context] **Leveraging Tactical Map Reconnaissance:** Military units are experts in terrain analysis and map reading. They already know *where* an enemy is likely to establish defensive ambushes, hide armor, or set up observation posts (OPs). AirScout lets them apply this existing tactical expertise directly—inputting suspected coordinates

into the dashboard to survey these high-risk Points of Interest (POIs) autonomously, keeping troops out of direct contact.

System Architecture & Data Flow

AirScout bridges high-level tactical autonomy with low-level PX4 flight control modes, utilizing ROS 2 Jazzy as the primary message-passing middleware and Gazebo Harmonic for physical simulation.

```
graph TD
    %% Subgraphs for separation
    subgraph Userspace ["Dashboard & Operator Control"]
        WebDash["Web Mission Cockpit Dashboard<br/>(HTTP @ Port 8080)"]
    end

    subgraph CustomROS ["Custom ROS 2 Package (scout_bot)"]
        TMC["Tactical Mission Coordinator Node<br/>(tmc_node.py)"]
        YOLO["YOLO Object Detection Node<br/>(yolo_node.py)"]
    end

    subgraph Bridges ["Bridges & Communications"]
        GZBridge["ros_gz_bridge<br/>(Gazebo-ROS Parameter Bridge)"]
        DDSAgent["Micro-XRCE-DDS Agent<br/>(UDP Port 8888)"]
    end

    subgraph Simulation ["PX4 Autopilot & Gazebo Simulator"]
        GZ["Gazebo Harmonic Simulator<br/>(Baylands Custom World)"]
        PX4["PX4 SITL Autopilot Firmware<br/>(uXRCE-DDS Client)"]
    end

    %% Data flow connections
    WebDash <-->|HTTP / API Requests & JSON Telemetry| TMC

    %% Sensor streams from Gazebo
    GZ -->|RGB Camera Frame| GZBridge
    GZ -->|Depth Camera Frame| GZBridge

    GZBridge -->|/camera/image_raw| YOLO
```

```

GZBridge -->|/camera/depth/image_raw| TMC

%% Detection signals
YOLO -->|/scout_bot/detections<br/>(Detection2DArray) | TMC

%% PX4 Status and telemetry
PX4 <-->|FastDDS Transmissions| DDSAgent
DDSAgent <-->|/fmu/out/vehicle_status_v4<br/>/fmu/out/
vehicle_local_position_v1| TMC

%% Flight commands from TMC
TMC -->|/fmu/in/offboard_control_mode<br/>/fmu/in/trajectory_setpoint<br/>
/fmu/in/vehicle_command| DDSAgent

%% Debug visualizer loop
YOLO -->|/scout_bot/debug_image<br/>(Watermarked HUD Frame) |
RViz["rqt_image_view / Debug Monitor"]

```

💡 What is Novel in This System? (Why you can't just "download and play" it)

If you download standard robotics or drone repositories, you generally get one of three things:

* **Low-level flight control** (e.g., PX4 or ArduPilot), which only understands how to fly to a coordinate in a straight line. * **Object detection code** (e.g., YOLO wrappers), which only outputs bounding boxes (pixel coordinates like $x=320$, $y=240$) on a video screen. *

Heavyweight navigation stacks (e.g., ROS 2 Nav2), which require massive computational power, active sensors (LiDAR), and pre-built 2D static maps of the environment.

Our Custom Synthesis

AirScout builds a custom **Tactical Brain** ([tmc_node.py](#)) that fuses these disconnected worlds into a singular tactical capability. Here is what is custom-coded and unique to our project:

- **Passive Depth-to-Flight Reactive Math (APF):** Unlike standard collision avoidance that stops the drone or requires a pre-mapped forest, our Artificial Potential Field (APF) algorithm splits raw depth images into vertical sectors and calculates vectors on the fly. It

doesn't just stop; it dynamically decides whether to strafe left, slide right, or climb over trees in the Z-dimension—all using passive camera inputs.

- **Trigonometric Visual Target Projection:** A standard camera feed only shows you *that* a vehicle is there. Our system uses the drone's altitude, current yaw heading, and the camera's fixed downward pitch angle (35°) to project those 2D pixels onto the 3D ground. It translates a visual detection directly into real-world geographic coordinates (MGRS), meaning the drone does the target localization math automatically.
- **Automated Tactical Flight Behaviors:** You cannot download "Recon by Fire" or "Investigate & Scan" modes. We mathematically coded these specific flight patterns:
 - *Scan Mode:* Uses the camera tilt angle to calculate the necessary orbit radius to keep the target dead-center while rotating the drone's nose. During this orbit, the camera stream is fed into YOLOv10 to classify threat classes (vehicles, personnel) and runs a 2.5-meter spatial clustering filter in the local frame. It tracks unique targets and tallies counts based on a high-water mark logic.
 - *Recon by Fire Mode:* Runs a lemniscate (figure-8) trajectory with vertical sinusoidal sweeps to intentionally draw fire or provoke a reaction.
- **Integrated, Lightweight Cockpit:** Typically, ROS projects require launching heavy UI software like RViz. Our system hosts a lightweight, zero-dependency HTTP server directly inside the ROS node, serving a custom web interface that works on any browser (including tablets or phones in the field).

Core Custom Functionality Detail

1. Custom MGRS Coordinate Parser & Frame Mapper

To interface with standard military coordinates, `tmc_node.py` features a custom Military Grid Reference System (MGRS) parser. Given an MGRS target string (e.g. `10SEG 88650 41222`), it:

1. Strips spacing, validates the grid zone designator (`10SEG`), and extracts easting and northing.
 2. Translates the coordinates into localized simulated ENU/NED coordinates relative to the Gazebo simulation origin (Easting = 88523.5 m, Northing = 41042.8 m).
 3. Computes the local position offsets (North and East) simply by subtracting the origin offsets from the target coordinate values.
-

2. 2D-to-3D Visual Target Projection

The camera is physically mounted on the drone with a fixed downward pitch angle of 35 degrees. When YOLOv10 detects targets, `tmc_node.py` projects the 2D bounding box center coordinates into the 3D local coordinate space without requiring an active 3D depth map:



- **Normalized Screen Position:** Determines where the bounding box center lies vertically on the screen (from -1 at the top to +1 at the bottom).
- **Pitch Angle Adjustment:** Calculates the angle offset of the detected object relative to the camera's optical axis based on the camera's vertical Field of View (~54 degrees).
- **Net Downward Angle:** Adds the camera's physical 35-degree tilt to the pixel angle offset to find the net angle from the horizon.
- **Ground Distance:** Computes the horizontal distance to the target using simple trigonometry (altitude divided by the tangent of the net downward angle).
- **Global Location:** Adds the calculated horizontal distance and direction (based on the drone's current compass heading) to the drone's coordinates to estimate the target's exact 3D coordinates on the ground.

3. Reactive Obstacle Avoidance via Artificial Potential Fields (APF)

To safely navigate forest canopies and steep topography without a heavy map-building node, AirScout implements a customized **3D Artificial Potential Field** algorithm operating directly on real-time depth frames:

- **3-Sector Depth Processing:** The raw depth sensor matrix is divided into Left, Center, and Right vertical columns. The minimum valid depth in each sector determines local distance readings.

- **Attractive Force:** Generates a vector pulling the drone directly towards its destination coordinates.
 - **Repulsive Forces:** Activates when obstacles enter the safety distance threshold (12 meters):
 - **Center Obstacle Avoidance:** Pushes backward and applies a dynamic lateral bias (steering left or right depending on which side has more clearance) to navigate around obstacles.
 - **Lateral Obstacle Avoidance:** Pushes the drone sideways away from nearby trees or objects on its left or right.
 - **Vertical Climbing Avoidance:** Dynamically increases flight altitude to climb over trees and rising terrain.
 - **Smoothing & Stabilization:**
 - **Exponential Moving Average (EMA) Filtering:** Smooths the direction vector over time (80% weight on new vector, 20% on previous) to eliminate sudden turns and control oscillations.
 - **Dynamic Velocity Scaling:** Automatically drops cruise speed from 12 m/s down to 4 m/s in tight spaces.
 - **Yaw Alignment Scaling:** Slows linear movements when the drone's heading differs significantly from the direction of travel, preventing blind strafing.
-

4. Offboard State Machine & Mission Doctrines

The Tactical Mission Coordinator features a robust, state-based offboard flight controller that executes three custom mission doctrines:

1. **Investigate & Scan Mode:** Flies to the target coordinate, descends to 10m Above Ground Level (AGL), and performs a circular orbit. The orbit radius (approximately 14.28 meters) is calculated to keep the target dead-center in the 35-degree pitched camera frame, and the drone continuously yaws to keep its nose aimed directly at the center.
2. **YOLO Target Classification, Clustering & Counts:** As the drone circles, the camera stream feeds into YOLOv10 to classify threat classes (vehicles, personnel). To prevent counting the same object multiple times from different angles, `tmc_node.py` converts 2D bounding boxes into physical 3D ground locations. Detections are grouped using a tight **2.5-meter spatial clustering filter**. A **high-water mark count** tracks the maximum number of targets of a single class visible within that cluster in a single video frame. Once the 360-

degree orbit completes, the node compiles a clean, consolidated threat report for the dashboard.

3. **Recon by Fire Mode:** Performs an aggressive decoy maneuver over the target area to provoke threat reactions. It traces a smooth figure-8 (lemniscate) swoop path centered over the target coordinate while oscillating its altitude between 8m and 18m.
4. **Observation Post Mode:** Flies to a standoff location 21 meters South of the target, hovers indefinitely at 15m AGL, and faces North directly towards the coordinates to monitor the area.
5. **Continuous Monitoring & Intel Reporting:** While in the hover position, the drone continuously runs YOLO detection on the coordinate area, projecting target positions and updating the contact database in real time. When the operator recalls the drone (RTL) or redirects it to a new objective, it automatically compiles and logs a consolidated intelligence report of all observed threat clusters.

5. Interactive Web Mission Cockpit Dashboard

An embedded HTTP server (`BaseHTTPRequestHandler`) runs in a background thread of `tmc_node.py` . It serves a custom dashboard with: * Real-time flight states, speeds, altitude (Z), and current MGRS coordinate readouts. * Live mission selection controls (MGRS target strings, Mission Mode selection, Execute and Recall/Abort buttons). * Real-time, color-coded Operations Logs. * Contact Intel Database listing identified threats (class, count, projected MGRS, timestamp, and confidence).

Force Protection & Tactical Rationale (Saving Lives)

AirScout is designed around real-world tactical viability and force protection. By solving key integration challenges in visual target mapping and collision avoidance, the system delivers direct advantages to units in the field:

- **Eliminating the "Fatal Funnel" (Recon by Proxy):** Traditional cavalry scouting requires sending soldiers in vehicles or on foot to clear blind spots, forested hills, or suspected ambush positions. AirScout acts as a forward scout—exploring high-risk areas and reporting back coordinates before any friendly troop enters a potential engagement zone.

- **Leveraging Core Military Competencies:** Instead of requiring weeks of training to master manual drone piloting, AirScout abstracts flight control entirely. Soldiers apply their existing map-reading and terrain reconnaissance skills: identifying likely enemy defensive locations on a topographic map, typing those MGRS coordinates into the dashboard, and letting the drone execute the search autonomously.
 - **Electromagnetic Resiliency (EW Hardening):** Standard obstacle-avoidance systems emit active laser pulses (LiDAR) or radio frequency signals (Radar) that can be intercepted by modern Electronic Warfare (EW) direction-finding platforms. AirScout relies entirely on passive RGB-D camera sensing and raw visual depth streams to navigate, keeping the drone (and the operating squad) electromagnetically silent.
 - **Dynamic Decoy Operations (Recon by Fire):** The "Recon by Fire" mode utilizes an aggressive, changing flight trajectory (figure-8 sweeps and altitude dips) to mimic manned aircraft or high-value targets. This tricks hidden adversaries into firing at an inexpensive, unmanned asset—revealing their defensive positions and weapon emplacements while keeping friendly forces concealed.
-

Authorship & Project Structure

All core tactical coordination, sensor calculations, state machines, and visual watermarking modules were custom-developed for this project.

- [ros_ws/src/scout_bot/scout_bot/tmc_node.py](#): The central brain of the UAV. Contains the MGRS converter, the visual coordinate projector, the 3D potential field obstacle avoidance math, the mission finite state machine, and the embedded HTTP dashboard server.
 - [ros_ws/src/scout_bot/scout_bot/yolo_node.py](#): Handles object detection inference on the simulated OakD-Lite RGB camera feed. It filters confidence thresholds, counts targets, and draws a watermark HUD overlaid on the published `/scout_bot/debug_image` topic.
 - [ros_ws/src/scout_bot/worlds/baylands_custom.sdf](#): Custom Gazebo environment containing custom-generated terrain heights, trees, hills, and target model configurations.
 - [start_all.sh](#): Master bash launcher which manages starting background processes, setups traps for proper process termination, and cleans up simulators upon exit.
-

Code Reuse & Dependencies

The AirScout architecture integrates several industry-standard libraries, frameworks, and packages. This integration leverages external software for simulation, lower-level control, and inference, freeing the custom packages to implement tactical decision-making and obstacle avoidance.

Dependency / Library	Component Reused	Integration & Purpose in AirScout
Pixi	Environment & Package Manager	Handles conda and PyPI dependencies, ensuring Apple Silicon native builds of ROS 2 and Gazebo.
ROS 2 Jazzy	Middleware & Communication Core	Provides the publisher/subscriber infrastructure and message classes (<code>sensor_msgs</code> , <code>vision_msgs</code>).
PX4 Autopilot SITL	Flight Controller & State Estimation	Simulates the drone's inner control loop, local position estimation, and command execution (arming, takeoff, offboard trajectory following).
Gazebo Harmonic	Physics & Sensor Simulator	Renders the 3D world, computes rigid-body dynamics, and generates RGB and depth camera images.
Micro-XRCE-DDS Agent	DDS Communication Bridge	Translates lightweight uXRCE messages from the PX4 firmware into DDS messages accessible to ROS 2.
ros_gz_bridge	Parameter Bridge	Passes camera and depth image topics from Gazebo Harmonic to ROS 2.
Ultralytics YOLOv10	Object Detection Network	Reuses pre-trained YOLOv10 weights (<code>yolov10n.pt</code>) to run fast, real-time object classification on the RGB video stream.
CvBridge & OpenCV	Computer Vision Libraries	Used in <code>yolo_node</code> and <code>tmc_node</code> to convert between ROS Image messages and numpy arrays for vision math.

Setup and Execution Guide

Follow these steps to configure, build, and run the project repository on macOS (compiled natively for Apple Silicon).

Prerequisites

Ensure you have the following installed on your machine: * macOS (M1/M2/M3 Silicon) * [Pixi Package Manager](#) (to manage the entire ROS 2, Gazebo, and PyTorch environment cleanly):

```
bash curl -fsSL https://pixi.sh/install.sh | bash
```

 (Restart your terminal after installing Pixi)

Step 1: Initialize Environment and Dependencies

Pixi reads the configuration in `pixi.toml` and automatically configures a local environment under `.pixi/`:

```
pixi install
```

Step 2: Build the ROS 2 Workspace

Use `colcon` inside the Pixi environment to build the custom `scout_bot` package:

```
pixi run colcon build --symlink-install
```

Step 3: Run the System

The repository includes a master orchestrator script `start_all.sh` that cleans up old simulations and spins up all five stages of the system in the background.

To start everything:

```
./start_all.sh
```

What happens under the hood:

1. **DDS Agent** starts (`./run_dds_agent.sh`), waiting for the PX4 SITL.

2. **PX4 SITL & Gazebo** launch (`./test_sim.sh`), loading the custom `baylands_custom` world and the depth camera-equipped `x500_depth_0` model.
 3. **ROS-Gazebo Bridge** starts (`./run_bridge.sh`), mapping sensor images into ROS 2 topics.
 4. **YOLO Detection Node** starts (`./run_yolo.sh`), subscribing to the camera.
 5. **TMC Node** starts (`./run_tmc.sh`), spawning the mission FSM and launching the web dashboard.
-

How to Operate the UAV

1. **Access the Dashboard:** Open your web browser and navigate to:
`http://localhost:8080`
2. **Enter Coordinates:** Input an target MGRS string in the box (e.g., `10SEG 88650 41222`).
3. **Select Mode:** Click on one of the three doctrine modes:
 4. *Investigate & Scan*
 5. *Recon by Fire*
 6. *Observation Post*
7. **Launch:** Click **Execute Objective**. The drone will arm, ascend to its hover altitude, fly enroute while dynamically climbing over trees, and perform the selected reconnaissance maneuver.
8. **Monitor Detections:** As the drone scans, any target vehicles or actors spotted by YOLO will be projected onto ground coordinates and populate the **Contact Report** table on the dashboard.
9. **Recall:** Click **Return Home/Recall** to abort a mission and order the drone to fly back to base and land automatically. Press `Ctrl+C` in the master terminal to clean up and close all background processes.